

Notes (Week 9 Monday)

These are the notes I wrote during the lecture.

Today:

- Total correctness Hoare logic
- Hoare logic + non-determinism

Then, player's choice:

1. Cover semantics of Hoare logic
a bit more slowly
2. Catch up on quiz reviews
3 votes!
3. Sneak peek at next Monday's topic: lambda calculus
4. Past exams (but that's planned for W10)

So far, we've done **partial** correctness Hoare logic

$\{\phi\} P \{\psi\}$

"P will never terminate with the wrong result"

If ϕ holds in the initial state, and we run P,
then **if** P terminates, then ψ holds.

Fun fact:

The program

while true do skip od

Satisfies every (partial correctness) Hoare triple.

$\{I \wedge b\} P \{I\}$

_____ (while)

$\{I\} \text{ while } b \text{ do } P \{I \wedge \neg b\}$

.....

_____ $\{\top \wedge \top\} \text{ skip } \{\top\}$

_____ While

$\{\top\} \text{ while true do skip } \{\top \wedge \perp\}$

_____ Conseq

$\{\top\} \text{ while true do skip } \{\perp\}$

We know that this Hoare triple is valid:

$$\models \{\top\} \text{ while true do skip } \{\perp\}$$

This is the weakest possible Hoare triple,
it implies every other Hoare triple
by the rule of consequence.

$$\frac{\varphi \Rightarrow \varphi' \quad \{\varphi'\} P \{\psi'\} \quad \psi' \Rightarrow \psi}{\{\varphi\} P \{\psi\}} \text{ (conseq)}$$

With partial correctness, we want a Hoare logic

$$[\varphi] P [\psi] \quad \text{total correctness Hoare triple}$$

total correctness

$$= \text{partial correctness (safety)} + \text{termination (liveness)}$$

If φ holds in the initial state,
then P *will* terminate, and the final state
satisfies ψ

Aside: the square-bracket notation
is not universally understood

$$\begin{array}{ll} \mathcal{L} := & x := e \quad \text{(assign)} \\ & | \quad P; Q \quad \text{(seq)} \\ & | \quad \text{if } b \text{ then } P \text{ else } Q \text{ fi} \quad \text{(if)} \\ & | \quad \text{while } b \text{ do } P \text{ od} \quad \text{(while)} \end{array}$$

$$\frac{}{[\varphi[e/x]] \ x:=e \ [\varphi]} \text{ (assign)}$$

Q: what does $\varphi[e/x]$ mean?

A: Notation for substitution

Replace all free occurrences of x with e
in φ

$$\varphi[x:=e] \quad \varphi\{e/x\} \quad \varphi[e/x]$$

$$\frac{[\varphi] P [\rho] \quad [\rho] Q [\psi]}{[\varphi] P; Q [\psi]} \text{ (seq)}$$

$$\frac{[\varphi \wedge b] P [\psi] \quad [\varphi \wedge \neg b] Q [\psi]}{} \text{ (if)}$$

$$[\phi] \text{ if } b \text{ then } P \text{ else } Q \text{ fi } [\psi]$$

$$\frac{\phi \Rightarrow \phi' \quad [\phi'] P [\psi'] \quad \psi' \Rightarrow \psi}{[\phi] P [\psi]} \text{ (conseq)}$$

While rule:

previously, we required an **invariant**
Invariant, in English: something unchanging
Something that's true when we start,
and stays true for every loop iteration.

To establish termination, we'll want a **variant**
Variant, in English: something changing.
Essentially the same thing as a measure.
An integer value computed from the state,
that decreases with every loop iteration,
and has a lower bound
(for convenience, we're using 0 as this bound)

$$\frac{[I \wedge b \wedge v = N] P [I \wedge v < N] \quad I \wedge b \Rightarrow v > 0}{[I] \text{ while } b \text{ do } P \text{ od } [I \wedge \neg b]} \text{ (while)}$$

Here, N is a constant (not a program variable)
doesn't have to be a specific number
v is an integer-valued expression
in the sense of predicate logic terms

```
[x, y = 0  ∧  N non-negative]
[y + (N-x)*M = N*M ∧ N non-negative ∧ x ≤ N]
while x < N do
  [y + (N-x)*M = N*M ∧ N non-negative ∧ x ≤ N ∧ x < N ∧ N-x = Z]
  x := x+1;
  y := y+M
  [y + (N-x)*M = N*M ∧ N non-negative ∧ x ≤ N ∧ N-x < Z]
od
[y + (N-x)*M = N*M ∧ N non-negative ∧ x ≤ N ∧ x ≥ N]
[y = N*M]
```

We need to add a variant to the above
x changes with every loop iteration, but increases
y may or may not change

We can pick:

v := N-x

```

[x,y = 0  ∧  N non-negative]
[...]
while x < N do
  [...] ∧ x < N ∧ N-x = Z
  [...] ∧ N-(x+1) < Z
  x := x+1;
  [...] ∧ N-x < Z
  y := y+M
  [...] ∧ N-x < Z
od
[...] ∧ x ≥ N
[y = N*M]

```

We end up needing to show

$N-x = Z \quad \text{implies} \quad N-(x+1) < Z$

That's not hard.

All that remains is the lower bound bit.

We need to show that invariant + loop guard true implies
that $N-x \geq 0$

We need to show that this:

$y + (N-x)*M = N*M \wedge N \text{ non-negative} \wedge x \leq N \wedge x < N$

Implies:

$N-x \geq 0$

The semantics of $P \in \mathcal{L}$, written $\llbracket P \rrbracket$,
is a relation between initial and final states.

This seems overkill for our purposes:

program in $\mathcal{L}$ are deterministic, so why not use
a partial function instead?

We didn't do this because having relations works
better with non-determinism.

You can think of it in two ways:

- there's more than one way for a program to execute
- there are parts of the program that are underspecified

Think non-determinism $\approx$ underspecification

For the non-determinism we'll introduce

a language $\mathcal{L}^+$ that does have non-determinism.

The intention behind this language is to allow
us to present "programs" that aren't necessarily
implementable as written, but that can be refined
into implementable programs.

- Underspecification (non-determinism)
- Magic (a command that magically makes things true)

```

 $\mathcal{L}^+ ::=$    $x := e$       (assign)
           |   $P; Q$         (sequential composition)
           |   $P + Q$       (choice, or "alternative composition")
           |   $P^*$         (repetition)
           |   $\varphi$       (assert)

```

φ think "assert φ "

this allows two interpretations:

- abort unless φ holds
- perform a miracle that makes φ true

under the second interpretation,
 the miracle represents a "hole" in our program,
 to make it implementable, we need to insert
 real code that actually establishes φ somehow,
 but we might want to abstract away from
 how exactly that's implemented

Repetition P^* means:

- run P zero or more times (analogous to Kleene star, or RTC)

$P + Q$ "do either P or Q "

- this is a program that non-deterministically chooses
 to behave either as P or Q , according to its own whims
- specifies a family of concrete programs, where
 in the implementation, the implementor can choose
 to either do P or Q

Roll a six-sided die

$(x:=1) + (x:=2) + (x:=3) + (x:=4) + (x:=5) + (x:=6)$

Q: When moving from $\mathcal{L}$ to $\mathcal{L}^+$, where did if and while go?

A: we don't particularly need them anymore

```

if b then P else Q      (b; P) + ( $\neg$ b; Q)
while b do P od         (b; P)*;  $\neg$ b

```

quicksort(xs):

```

choose a pivot  $x \in xs$ ; // non-determinism
ys := everything  $\leq x$  in xs;
zs := everything  $> x$  in xs;
return(quicksort(ys) + quicksort(zs))

```

The point: any concrete implementation needs to have an
 actual specified, executable way to choose a pivot.

But: for the purposes of reasoning about whether quicksort is correct (in the sense that it sorts), we don't care. **Any** method of choosing the pivot leads to a correct sorting algorithm.

By modelling "choose a pivot" as a non-deterministic choice, we can verify many possible quicksort algorithms at the same time.

Q: What's the semantics of $\mathcal{L}^+$?

$$\llbracket P + Q \rrbracket = \llbracket P \rrbracket \cup \llbracket Q \rrbracket$$

$$\llbracket P^* \rrbracket = \llbracket P \rrbracket^* \quad (\text{note the first } * \text{ is program syntax, the second star is reflexive-transitive closure of relations})$$

$$\llbracket \varphi \rrbracket = \langle \varphi \rangle$$

$$\langle \varphi \rangle = \{(s, s) \mid \llbracket \varphi \rrbracket s\}$$

Q: What are the Hoare rules?

$$\frac{\{I\} P \{I\}}{\{I\} P^* \{I\}} \quad (\text{repetition})$$

$$\frac{\{ \varphi \} P \{ \psi \} \quad \{ \varphi \} Q \{ \psi \}}{\{ \varphi \} P + Q \{ \psi \}} \quad (\text{alternative})$$

(The rule for asserts: saved for the quiz)

$$(x > y) [x := x + y]$$

$$\equiv x + y > y$$

$$\equiv x > 0$$

$$\{(x > y) [x := x + y]\} x := x + y \{x > y\}$$

A transition system is a tuple $(S, \rightarrow)$

– S is a set of states

– $\rightarrow \subseteq S \times S$

An invariant is something whose truth is preserved by every transition.

Observation: $\top$ is an invariant of every transition system.

Some optional components:

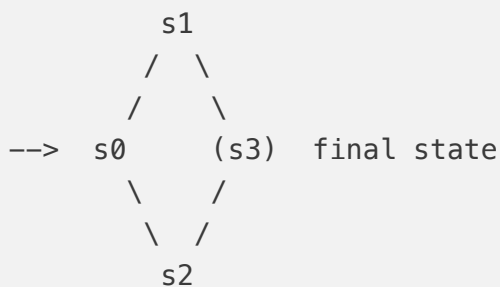
– A start state

- A set of final states
- Transition labels

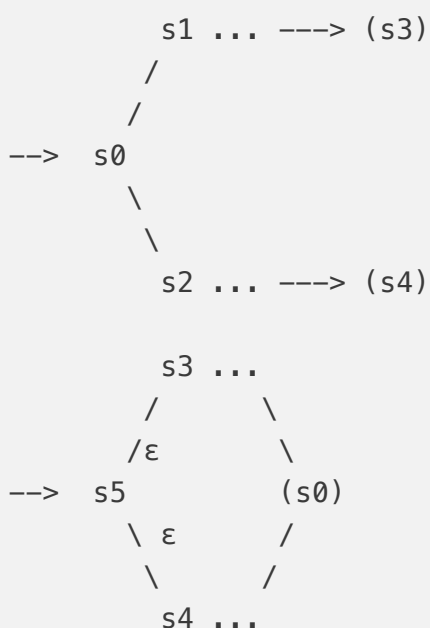
Here's a transition system:

$(\{s_0, s_1\}, s_0, \emptyset)$

A non-deterministic transition system is one where some state have more than one successor state



Suppose L is a regular language, with the following NFA:



Intuition: you can make an NFA that accepts the reverse language, by reversing all arrow, making the initial state final, and adding an initial state with ϵ -transitions to all previously final states